Routability-Driven PCB Placement and Congestion-Aware Detailed Routing: Prior Art and Architectural Recommendations for Autonomous KiCad Toolchains

The plateau observed in the autonomous automotive PCB flow—where agent-ee packs 309 footprints across 17–19 functional blocks in 2–3 minutes, yet kicad-route stalls deterministically at 203 of 235 nets (86.4% completion) and degrades to 171–176 routed nets (59–64 open) when unrouted nets are promoted in subsequent campaign rounds—is a textbook manifestation of two decoupled failure modes in physical design automation. First, the two-level deterministic first-fit packer optimizes purely for pad-to-pad decoupling locality and minimum block bounding-box area without modeling **routing track supply**; packing eFuse setting resistors at 0.26 mm courtyard gaps creates a physical copper-to-copper channel narrower than the $0.60\ \text{mm}$ minimum required to pass a single $0.2\ \text{mm}$ track with $0.2\ \text{mm}$ clearances on F.Cu, while also leaving insufficient clearance to drop a via to B.Cu[1]. Second, running a single-pass A* grid router with rip-up disabled and re-ordering failed nets to the front of later campaigns induces **greedy channel capture**: when previously blocked nets (including long inter-block buses) route first on an empty board with zero congestion history, they cut straight across high-value pin-escape corridors, starving 59–64 downstream nets that previously succeeded[2].

The following analysis examines prior art across IC and PCB physical design literature, open-source repositories, and industrial routers across five core domains, concluding each section with ranked, minimal-diff architectural modifications for agent-ee and kicad-route.

1. Routability-Driven Placement

Cheap Congestion Estimators in Placement

In both VLSI and PCB placement, invoking a full maze router inside the inner placement loop is computationally prohibitive, motivating fast geometric and topological proxies of routing demand[4]. The foundational geometric proxy is **Rectangular Uniform Wire Density (RUDY)**, introduced by Spindler and Johannes (DATE 2007, *"Fast and Accurate Routing Demand Estimation for Efficient Routability-driven Placement"*)[4]. RUDY models the routing demand of each net n (or hyperedge e) as a uniform wire density distributed across its axis-aligned bounding box $w_n \times h_n$, where $w_n = x_n^{\max} - x_n^{\min}$ and $h_n = y_n^{\max} - y_n^{\min}$ [4]. Given an estimated wirelength L_n (typically the Half-Perimeter Wirelength,

$\mathrm{HPWL}_n = w_n + h_n$) and wire pitch or width p, the per-net wire density is defined as[4]:

$$d_n = \frac{L_n \cdot p}{w_n \cdot h_n} = \frac{\text{Wire Area}}{\text{Net Area}}$$

For any point (x, y) or evaluation bin g, the cumulative RUDY demand is the superposition of all nets overlapping that location, $\mathrm{RUDY}(g) = \sum_{e \in E} \mathrm{OA}(e, g) \cdot d_e$, where $\mathrm{OA}(e, g) \in [0, 1]$ is the fractional overlap area of net e's bounding box with bin g[4]. In modern GPU-accelerated analytical placers such as DREAMPlace (limbo018/DREAMPlace, dreamplace/ops/rudy/) and machine-learning surrogates, this is frequently simplified to a unit-pitch proxy $\sum_e (w_e + h_e)/(w_e h_e)$ or the sum of inverse net bounding-box areas $\sum_e (w_e h_e)^{-1}$[7]. RUDY's primary advantage is that its per-net density calculation depends neither on an arbitrary bin-grid alignment nor on a specific router's tree topology[4].

Because standard RUDY dilutes routing demand across the entire bounding box of long nets, it systematically under-predicts localized pin-access bottlenecks around high-degree components[7]. To correct this, **PinRUDY**—formalized in DREAMPlace-Cong (Liu et al., DATE 2021) and open-source congestion predictors (ycchen218/EDA-Congestion-Prediction)—defines a pin-density map that weights each individual pin $p \in e$ by its incident net's RUDY density d_e (or $1/(w_e h_e)$) inside the specific bin b_{ij} containing pin p[7]. A second limitation of baseline RUDY inside DREAMPlace is the assumption that wires can pass unimpeded over macros[5]. In **AutoDMP** (Agnesina et al., ISPD 2023, NVlabs/AutoDMP), RUDY is extended into **RUDY with Macro Blockages** to account for obstructions that force nets to detour around macro perimeters[5]. Given horizontal and vertical gcell routing supplies $s(g)_{H/V}$ derived from the number of routing layers and minimum metal pitches, AutoDMP defines a per-macro unit routing demand $\alpha(m)_{H/V} = o(m)_{H/V} / a(m)$ —where $a(m)$ is the area of macro m and $o(m)_{H/V}$ is the routing supply consumed by macro m's internal obstructions—and adds this macro blockage demand directly into the RUDY utilization map[5].

Rather than spreading wire density uniformly across a bounding box, Steiner-tree-based estimators construct a Rectilinear Steiner Minimal Tree (RSMT) for each multi-pin net using **FLUTE** (Chu, ICCAD 2004 / TCAD, *"FLUTE: Fast Lookup Table Based Wirelength Estimation Technique"*), which employs precomputed lookup tables for low-degree nets and net-breaking heuristics for higher-degree nets[5]. Used in placers and routers such as Kraftwerk, AutoDMP, and CUGR (InitRoute.cpp), FLUTE provides exact minimal rectilinear wirelength[5]. However, as Spindler and Johannes demonstrated, a multi-pin net possesses multiple degenerate RSMTs of

identical minimal length (such as six distinct minimum-length Steiner trees for a 6-pin net), making single-tree demand maps vulnerable to local quantization uncertainty when a detailed router detours around an obstacle[4]. At the highest fidelity tier, **global-route-in-the-loop** placers (Rooster, SimPLR, Ripple, POLAR 2.0, and RUPlace) periodically invoke a fast pattern-and-maze global router—most commonly **FastRoute** (Xu, Zhang, & Chu, TCAD) or **NCTU-GR**—once placement density overflow drops below a predefined threshold, replacing geometric proxies with actual gcell edge demand $d(e)$ and capacity $c(e)$ at the expense of higher runtime[6].

Congestion Estimator	Primary Source / Implementation	Computational Complexity	Captures Pin-Access Bottlenecks?	Captures 2-Layer Planar Crossings?	Suitability for Discrete Macro Packer (agent-ee)
Standard RUDY	Spindler & Johannes (DATE 2007); DREAMPlace (ops/rudy)[4]	$\mathcal{O}(\|E\|)$ per evaluation	Poor (diluted over net bbox)[7]	No (bbox invariant to pin swap)[16]	Low for intra-block; moderate for inter-macro channel heatmaps
PinRUDY	Liu et al. (DATE 2021); ycchen218/EDA-Congestion-Prediction [cite: 7, 10]	$\mathcal{O}(\|P\|)$ per evaluation	High (localizes net weight at pins)[7]	No	**High** for pre-sizing component courtyards before packing
RUDY + Macro Blockages	Agnesina et al., NVlabs/AutoDMP (ISPD 2023)[5]	$\mathcal{O}(\|E\| +$	Moderate	No	**High** for evaluating rigid macro arrangements
FLUTE RSMT	Chu (TCAD 2008);	$\mathcal{O}(\|E\| \cdot$	Moderate	Only if combined	Moderate (decompos

Demand	cuhk-eda/cu-gr (InitRoute.cpp)[5]	via LUT		with intersection checks	es multi-pin nets into 2-pin edges)
Net Crossing (NC) + Cascaded Spacing	Zhang et al., NVlabs/Cypress (ISPD 2025); Gu et al. (ICCAD 2025)[16]	$\mathcal{O}(\|\|E_{2\text{-pin}}\|\|^2$ (or $\mathcal{O}(N \log N$ sweep)	High (when combined with pad halos)	**Yes** (directly counts topological conflicts)[16]	**Very High** for both intra-block orientation/ order and macro placement
Global-Route-in-the-Loop	FastRoute 4.0 / RUPlace (DAC 2025)[14]	$\mathcal{O}(\|\|E\|\| \cdot$	High	Yes (on coarse grid)	Too slow for inner backjumping; ideal between packing and routing

Integration into Placer Objectives and Applicability to Constructive Macro Packing

Routability estimators are integrated into placement engines through either direct objective penalization or indirect supply allocation[4]. In analytical gradient-based placers (DREAMPlace, Cypress, PPAPlace), congestion is injected directly as a differentiable penalty term into the global objective $\min_{\mathbf{x},\mathbf{y}} \mathcal{L}(\mathbf{x},\mathbf{y}) = \mathcal{W}(\mathbf{x},\mathbf{y}) + \lambda_D \mathcal{D}(\mathbf{x},\mathbf{y}) + \lambda_C \mathcal{L}_{\text{cong}}(\mathbf{x},\mathbf{y})$ [7]. Because step-function bin overlaps in RUDY and PinRUDY are not differentiable everywhere, DREAMPlace-Cong and recent differentiable surrogate placers smooth net bounding boxes via log-sum-exp (LSE_γ) approximations and backpropagate subgradients $\nabla_x \mathcal{L} = J_x(\text{RUDY})^T \nabla_{\text{RUDY}} \mathcal{L} + J_x(\text{PinRUDY})^T \nabla_{\text{PinRUDY}} \mathcal{L}$ (or Wasserstein distance gradients in **RUPlace**, DAC 2025) to push cells out of congested hotspots[7]. Conversely, the indirect approach—pioneered by Brenner and Rohe (ISPD 2002 / 2003, *"An Effective Congestion-Driven Placement Framework"*) and refined in Kraftwerk, SimPLR, POLAR 2.0, and RUPlace—uses **cell inflation** to scale the physical area or courtyard halo of components in congested regions by an inflation ratio $l_g > 1$ [4]. When the placer's density-spreading or legalization step enforces non-overlap on the inflated footprints, it naturally spreads components apart, reducing local pin density and increasing routing track supply[4].

Because agent-ee employs a deterministic first-fit constructive search with backjumping rather than continuous gradient descent, differentiable RUDY subgradients do not apply directly. Instead, cell inflation and discrete topological cost pruning map natively to a constructive two-level packer. Prior to executing the intra-block first-fit search, agent-ee can compute the signal pin count and local pass-through net demand of each footprint and inflate its packing courtyard asymmetrically along pin-bearing edges[14]. During the constructive backjumping search itself, each candidate placement or orientation of a part (at the intra-block level) or rigid macro (at the board level) can be evaluated in microseconds using 2-pin segment intersection counts and boundary channel supply-vs-demand checks, triggering immediate backjumping whenever a candidate packing state traps an unescaped pin[16].

How PCB-Specific Placers Differ from IC Placers (2018–2026)

Recent physical design literature demonstrates that VLSI congestion estimators fail when transplanted naively to PCBs[16]. In **Cypress** (Zhang, Agnesina, Shbat, Leader, Zhang, & Ren, ISPD 2025 Best Paper, *"Cypress: VLSI-Inspired PCB Placement with GPU Acceleration"*, open-sourced at github.com/NVlabs/Cypress, with continuous orientation relaxation also adopted in github.com/Studio-Fug/yapnr), the authors prove that RUDY is a flawed proxy for PCB routability[16]. In VLSI placement, 8–15 orthogonal metal layers allow multi-pin nets to route within their bounding boxes across vertical layers[16]. On a PCB with far fewer routing layers—such as two signal layers (F.Cu and B.Cu)—planarity dominates: as illustrated in Figure 3 of the Cypress paper, two distinct net-pair configurations connecting two 2-pad SMD components inside the exact same bounding box produce identical RUDY congestion scores whether the two pin-to-pin connections run parallel (zero routing conflict) or intersect in an X-crossing (creating a direct routing conflict that forces a layer-switching via or a long perimeter detour)[16]. Rotating one of the two footprints by 180° untangles the X-crossing into parallel connections without changing the bounding-box RUDY score at all[16]. Cypress therefore replaces RUDY with an explicit **Net Crossing (NC)** penalty in its global placement loss $\mathcal{L} = \mathcal{W}(x, y, \theta) + \lambda_D \mathcal{D}(x, y) + \lambda_{NC} NC(x, y, \theta)$, reducing net crossings by up to 73% and boosting routability by up to $5.9\times$ [16]. Extending this topological formulation, Gu et al. (ICCAD 2025) model the additional spacing needed for cascaded net crossings directly as an area congestion demand $R_{\text{cong}} = \#CR_{\text{net}} \times S_{\text{crs}}$, multiplying the estimated crossing count $\#CR_{\text{net}}$ by the physical area footprint of a single crossing S_{crs} [17].

Furthermore, unlike IC standard cells constrained to fixed rows and upright/flipped sites, PCB components exhibit diverse aspect ratios and can rotate to any legal orientation $\theta_c \in \Theta$ [16]. Cypress incorporates an orientation-aware log-sum-exp wirelength model where pin coordinates (x_k, y_k) are rotated around component center (x_c, y_c) via $x_k = x_c + x_k^{\text{off}} \cos\theta_c - y_k^{\text{off}} \sin\theta_c$ and $y_k = y_c + x_k^{\text{off}} \sin\theta_c + y_k^{\text{off}} \cos\theta_c$, using Gumbel-Softmax reparameterization $\kappa_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum \exp((\log \pi_i + g_i)/\tau)}$ in a bilevel

optimization loop that alternates between updating positions (x, y) and orientation probabilities π [16].

Across the broader ecosystem of published and open-source tools referenced in industry discussions, important architectural distinctions apply. **DeepPCB** (launched by InstaDeep in November 2019, with its KiCad plugin open-sourced in May 2026 at github.com/instadeepai/deeppcb-kicad-plugin) is strictly a reinforcement-learning autorouter rather than a component placer; it ingests a fixed board placement and routes traces using RL policies trained on environments such as InstaDeep's Jumanji Connector and Sable[23]. (Note: The exact internal neural weights and reward shaping of DeepPCB's proprietary cloud engine are not publicly disclosed, though its open-source grid benchmark distribution is replicated in PCBWorld-Bench[23]). Similarly, **Horizon EDA** (github.com/horizon-eda/horizon) does not implement an automated placement algorithm, operating instead as a unified C++/GTKmm/OpenGL schematic and board editor that embeds a KiCad-derived interactive router[28]. Meanwhile, **Google-style RL macro placers** (CircuitTraining, MaskPlace, and WireMask-EA at NeurIPS 2023) place macros sequentially on a grid canvas guided by a wirelength mask and the mean RUDY score of the top 10% most congested grid cells, $\frac{1}{|G^*|}\sum_{g_i \in G^*} \mathrm{RUDY}(g_i)$ [29]. Crucially, the upstream **KiCadRoutingTools** repository (drandyhaas/KiCadRoutingTools, Haas 2026) from which kicad-route branched at v0.23.1 has recently introduced placement-routing co-design modules—specifically py_router/group_routing.py, py_router/place_motion.py, tests/stress/demand_halo_study.py, tests/stress/block_relocation_study.py, tests/stress/measure_834_835_side_awareness.py, and tests/stress/measure_836_flip_vs_reseat.py—which evaluate flipping a passive by 180° or reseating it versus relocating entire functional blocks when intra-block routing fails[31]. (Note: Full line-by-line source code for KRT's internal placement/groups.py in the 2026 bus622-take5 branch could not be verified from public index snapshots and is flagged accordingly[32]). Finally, PCB placers must account for through-hole pins and plane layers: plated through-hole (PTH) connector pins and non-plated mounting holes (NPTH) act as all-layer blockages across F.Cu, In1, In2, and B.Cu, whereas internal plane layers (In1=GND, In2=+3V3) absorb power and ground connections vertically through local fanout vias, eliminating horizontal XY wirelength and congestion demand for GND and +3V3 nets[24].

What This Would Mean for the In-House Tools

The highest-payoff, lowest-cost improvement for agent-ee (~40 lines of Python in the intra-block packer) is adding a **2-Pin Net-Crossing and Orientation Check** directly inside the deterministic first-fit backjumping objective while excluding GND and +3V3 plane nets[16]. Whenever agent-ee places a 2-pin passive (such as an eFuse setting resistor) near its parent IC, it should evaluate candidate rotations ($0^\circ, 90^\circ, 180^\circ, 270^\circ$) and count segment intersections among intra-block signal connections, immediately rejecting or penalizing orientations that create a planar X-crossing or force a ratsnest segment across an adjacent pad's courtyard[16]. As a second-priority change (~80 lines of Python in the macro-level

arranger), agent-ee should replace pure bounding-box area minimization when arranging the 17–19 rigid functional blocks with a composite **HPWL + Inter-Macro Net-Crossing + Macro-Blockage RUDY score** (following Cypress and AutoDMP), registering fixed mechanical obstacles (the mounting holes, IMU keep-out ring, and RF corridor) with unit blockage demand $\alpha(m) = 1.0$ so macros automatically orient and position themselves to untangle inter-block buses[5].

2. Channel Reservation Around Dense Macros

Concrete Formulas for Channel Width and Halo Sizing

In both IC floorplanning (such as Cadence Innovus addHaloToBlock and physical design channel-sizing rules) and PCB cluster packing, the minimum whitespace channel width W_{chan} between two adjacent macros or around a tight component cluster is governed by the fundamental **routing track supply vs. demand equation**[1]:

$$W_{\text{chan}} = \left\lceil \frac{N_{\text{pins,eff}} + N_{\text{pass}}}{\eta \cdot L_{\text{eff}}} \right\rceil \cdot P_{\text{track}} + S_{\text{margin}}$$

In this formulation, the routing pitch is $P_{\text{track}} = w_{\text{track}} + s_{\text{clear}}$, which equals $0.20\ \text{mm} + 0.20\ \text{mm} = 0.40\ \text{mm}$ on the target JLCPCB automotive board[1]. To pass k parallel tracks between two copper features, the geometry requires k track widths and $k+1$ clearances, yielding a total copper-to-copper span of $k(w_{\text{track}} + s_{\text{clear}}) + s_{\text{clear}}$; consequently, the boundary clearance margin is $S_{\text{margin}} = s_{\text{clear}} = 0.20\ \text{mm}$ (plus one router grid step of discretization margin if pads are not grid-aligned)[1]. The numerator sums the effective escaping signal pins $N_{\text{pins,eff}}$ along the facing macro edges (explicitly excluding GND and +3V3 pins that terminate locally into In1 and In2) and the estimated pass-through nets N_{pass} whose bounding boxes traverse the channel[1]. The denominator scales the available signal routing layers L_{eff} by a via-blockage discount factor $\eta \in [0.65, 0.80]$. Crucially, while $L_{\text{eff}} = 2$ (F.Cu and B.Cu) in wide inter-macro channels where signals can drop vias to the bottom layer, $L_{\text{eff}} = 1$ **inside tight SMD intra-block gaps** whenever the inter-component spacing is narrower than the $0.90\ \text{mm}$ copper-to-copper span required to place a standard $0.50\ \text{mm}$ via with $0.20\ \text{mm}$ clearances on both sides[1].

Root-Cause Geometry of the 0.26 mm Courtyard Gap Failure

Applying the exact copper-to-copper channel supply formula

$G_{\text{copper}}(k) = k \cdot w_{\text{track}} + (k+1) \cdot s_{\text{clear}} = 0.40k + 0.20\ \text{mm}$ to the target board's $0.20\ \text{mm}$ design rules reveals precisely why the eFuse setting resistors cannot be routed at a $0.26\ \text{mm}$ courtyard gap and why a global $0.60\ \text{mm}$ courtyard expansion fails[1]:

Routing Feature to Pass Between Two Pads	Copper-to-Copper Gap Formula	Exact Copper Gap Required (w=0.2,s=0.2 mm)	Required Courtyard Gap (assuming 0.15 mm IPC courtyard overhang per part)	Required Courtyard Gap (if courtyard equals pad edge)	Status at Current 0.26 mm Courtyard Gap
0 Tracks (Abutment Legal)	s_{clear}	$0.20\ \text{mm}$	$0.00\ \text{mm}$	$0.20\ \text{mm}$	Passes DRC, **0 tracks can pass**
1 Track on F.Cu ($k = 1$)	$1 \cdot w + 2 \cdot s$ [cite: 1]	$0.60\ \text{mm}$	$0.30\ \text{mm}$ (plus $1\times$ grid step ≈ 0.35–0.4)	$0.60\ \text{mm}$	**Blocked** (cannot pass 0.2 mm track)
1 Standard Via to B.Cu ($d_{\text{via}} = 0.50$)	$d_{\text{via}} + 2 \cdot s$	$0.90\ \text{mm}$	$0.60\ \text{mm}$	$0.90\ \text{mm}$	**Blocked** (cannot drop via to B.Cu)
2 Tracks on F.Cu ($k = 2$)	$2 \cdot w + 3 \cdot s$ [cite: 1]	$1.00\ \text{mm}$	$0.70\ \text{mm}$	$1.00\ \text{mm}$	**Blocked**
3 Tracks on F.Cu (	$3 \cdot w + 4 \cdot s$ [cite: 1]	$1.40\ \text{mm}$	$1.10\ \text{mm}$	$1.40\ \text{mm}$	**Blocked**

$k = 3$)					

When two passive footprints are packed at a $0.26\ \mathrm{mm}$ courtyard gap, the physical channel supplies $k = 0$ routing tracks on F.Cu and zero via slots to transition to B.Cu. Any third pad or IC pin trapped behind that gap is topologically unreachable on both signal layers. Conversely, widening all packing gaps globally to $0.60\ \mathrm{mm}$ fails because fixed mechanical structures (the connector $0.36\ \mathrm{mm}$ from mounting holes, the IMU keep-out ring, and the $0.16\ \mathrm{mm}$ RF corridor) and adjacent passive edges that share a net or require zero pass-through tracks ($k = 0$) only need the $0.20\ \mathrm{mm}$ bare copper clearance to pass KiCad DRC.

Selective Halo and Inflation Heuristics in Prior Art

Published placement flows avoid global spacing blowup by sizing halos selectively according to local pin and routing demand. In **AutoDMP** (NVlabs/AutoDMP, ISPD 2023) and **Cypress** (ISPD 2025), movable macros are temporarily enlarged by a macro halo during global placement so they cannot abut or trap standard cells, and those halos are subsequently removed or relaxed during final legalization so components can pack cleanly beside fixed keep-outs[5]. In **RUPlace** (Chen, Mai, & Lin, DAC 2025), components are grouped via modularity-based clustering (merging leaf nodes within four row heights when modularity gain is positive), and the placer computes the pin-sharing ratio $p(e, g)/|e|$ —the number of pins shared by net e and cluster g divided by the total pins in net e —to solve a convex program for each cluster's optimal inflation ratio l_g [14]. For hierarchical macro floorplanning, standard practice assigns an **asymmetric edge-specific halo** $H(B_i, s)$ to each of the four sides $s \in \{\mathrm{N}, \mathrm{S}, \mathrm{E}, \mathrm{W}\}$ of a functional block B_i proportional to the number of external signal nets $N_{\text{ext}}(B_i, s)$ exiting across side s [1]:

$$H(B_i, s) = \frac{1}{2}\left(\left\lceil \frac{N_{\text{ext}}(B_i, s)}{\eta \cdot L_{\text{eff}}} \right\rceil \cdot P_{\text{track}} + S_{\text{margin}}\right)$$

When two functional blocks B_i and B_j are packed adjacent to one another, the sum of their facing edge halos $H(B_i, \mathrm{E}) + H(B_j, \mathrm{W})$ automatically reserves the exact channel width required by their combined escaping pins, while macro edges facing a board keep-out or mounting hole with zero escaping pins receive $H(B_i, s) = 0$ [1].

What This Would Mean for the In-House Tools

The highest-payoff, lowest-cost modification (~35 lines of Python in agent-ee's intra-block packer) is replacing the uniform $0.26\ \text{mm}$ courtyard spacing with a **Pad-Edge Demand Halo**. agent-ee should retain the tight $0.16\text{–}0.26\ \text{mm}$ courtyard gap ($k = 0$ tracks) for fixed mechanical keep-outs, RF corridors, non-pad component body edges, and passive pads that connect exclusively to GND, +3V3, or the immediately adjacent component pad. However, on any pad-bearing edge of a multi-pin IC (such as the eFuse) or any passive pad whose signal net must route *past* a neighboring footprint, agent-ee should inflate that specific pad edge's halo so that the **copper-to-copper gap** to the next component is at least $G_{\text{copper}}(1) = 0.60\ \text{mm} + 1\ \text{grid step}$ ($0.65\text{–}0.70\ \text{mm}$ copper-to-copper)[1]. This single rule guarantees a 1-track escape channel on F.Cu for the eFuse setting resistors while leaving the $0.36\ \text{mm}$ connector-to-hole and $0.16\ \text{mm}$ RF corridors completely untouched. Second in priority (~50 lines of Python in agent-ee's macro packer) is attaching the **Asymmetric Edge Pin-Flux Halo** $H(B_i, s) = 0.20\ \text{mm} \times \lceil N_{\text{ext}}(B_i, s)/1.5 \rceil +$ to each of the four sides of the 17–19 packed functional blocks (counting only inter-block signal nets and ignoring GND/+3V3) so that inter-block bus channels scale automatically with pin flux[1].

3. Congestion-Aware Detailed Routing

Negotiated-Congestion Routing (PathFinder)

Sequential maze routers (Lee/A*) fail on dense boards because early nets route along shortest geometric paths with zero awareness of unrouted nets, creating permanent blockages that starve later nets regardless of net-ordering heuristics[2]. **PathFinder**, introduced by McMurchie and Ebeling (ACM FPGA 1995, *"PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs"*, pp. 111–117; US Patent 7,134,112 B1), resolved this order-dependence through **iterated negotiated congestion**[40]. Instead of treating existing routes as hard ∞-cost obstacles, PathFinder permits routing resources (grid nodes or edges n) to be shared or overridden during an iteration and rips up and reroutes nets across passes under the node cost function[40]:

$$c_n^{(k)} = \left(b_n + h_n^{(k)}\right) \cdot p_n^{(k)}$$

Here, b_n is the intrinsic base cost of node n (wirelength step or via cost), $p_n^{(k)} = 1 + p_{\text{fac}} \cdot \max(0, \text{occ}(n) - \text{cap}(n))$ is the **present congestion penalty** scaling with the number of nets currently contending for node n, and $h_n^{(k)}$ is the **historical**

congestion cost accumulated monotonically at the end of every pass k in which node n experiences contention[40]:

$$h_n^{(k+1)} = h_n^{(k)} + h_{\text{fac}} \cdot \max(0, \text{occ}(n) - \text{cap}(n))$$

The permanent memory in $h_n^{(k)}$ prevents cyclic oscillation: if a long bus with multiple viable detours and a trapped eFuse resistor pin with only one narrow escape corridor both contend for grid cell n, ripping them up without $h_n^{(k)}$ causes them to displace each other endlessly[23]. Because $h_n^{(k)}$ rises after every pass where n is over-subscribed, the price of traversing n eventually exceeds the extra wirelength cost of the bus's detour around the block, whereas the trapped intra-block net has no alternative path and pays the high price at n, converging to a legal assignment[40].

Global Routing, Rip-Up and Reroute, and History Costs in Modern Open Routers

Modern open-source global and detailed routers structure their search loops around PathFinder's history principle while adapting it to multi-layer grid graphs and geometric design rules. **CUGR** (cuhk-eda/cu-gr, Liu et al., DAC 2020 / TCAD) and **Dr. CU 2.0** (Chen et al., TCAD 2020) decompose multi-pin nets into 2-pin subnets via FLUTE (InitRoute.cpp), execute L-shape pattern routing to establish an initial congestion map, run three outer iterations of 2D maze rip-up and reroute (RRR), perform dynamic-programming layer assignment, and finish with region-based 3D RRR on 10×10 -GCell clips[13]. During A* search, CUGR weights the wirelength transition between GCell i and $i+1$ by a five-part cost function[18]:

$$\text{cost}_{\text{tot}}(i) = 1 + w_1 \cdot \text{cost}_{\text{cong}}(i) + w_2 \cdot \text{cost}_{\text{hist}}(i) + w_3 \cdot \text{cost}$$

where present congestion uses a smooth logistic curve that rises sharply as demand approaches supply, and history cost scales present congestion by an overflow counter[18]:

$$\text{cost}_{\text{cong}}(i) = \frac{\text{demand}(i)/(\text{supply}(i)+1)}{1 + e^{\text{supply}(i) - \text{demand}(i)}}, \quad \text{cost}_{\text{hist}}(i) = \text{h}$$

In CUGR, $\text{histCnt}(i)$ increments each time an overflow occurs on GCell i and is multiplied by a fractional decay factor (< 1.0) after each outer pass, while component weights are

calibrated to satisfy $w_1\text{cost}_{\text{cong}} < w_2\text{cost}_{\text{hist}} < w_4\text{cost}_{\text{overflow}} \ll w_3\text{cost}_{\text{block}}$[18].
Similarly, **FastRoute 4.0** (OpenROAD/src/grt, Xu et al., TCAD), **NCTU-GR 2.0** (Liu et al., TCAD 2013), and **SPRoute** combine fast pattern routing with iterative maze RRR where historical overflow costs accumulate on GCell edges and **virtual capacity adjustment** preemptively lowers the capacity parameter of congested edges to divert flexible nets before hard overflow occurs[15].
At the detailed routing level, **TritonRoute** (OpenROAD/src/drt, Kahng, Wang, & Xu, TCAD 2021, pp. 547–559) operates directly on a 3D fine-grained grid graph partitioned into 7×7 GCell worker clips (FlexDR_maze.cpp) and replaces coarse edge capacity with two soft geometry cost fields: **object cost** (object_cost) and **marker cost** (marker_cost)[46]. Rather than treating existing routed traces of other nets as hard obstacles during RRR, TritonRoute surrounds every routed wire and via with an object_cost halo; A* is allowed to route right across an existing net by paying object_cost, which creates a localized DRC violation[46]. At the end of the worker pass, the internal geometry DRC engine flags every short or spacing violation with a **DRC marker** and attaches a persistent marker_cost to that 3D bounding box[46]. In the next RRR iteration, only the nets touching a DRC marker are ripped up and rerouted, while marker_cost remains on the grid (and accumulates on repeated violations), forcing the more flexible net to detour around the contested spot[46].
By contrast, **Freerouting** (freerouting/freerouting, BatchAutorouter.java and AutorouteEngine.java) routes one connection at a time using octilinear room expansion (ShapeSearchTree), allowing A* to cross existing traces at a flat ripup_cost and immediately ripping up the crossed trace[23]. Because Freerouting makes purely local rip-up decisions without a persistent spatial history map or global lookahead, ripping up Net B to route Net A on a dense board causes Net B to rip up Net C, which in turn rips up Net A, resulting in cyclic board states that stall at 60–75% completion and trigger Freerouting v2.5.0's board-hash stagnation check[23]. Notably, in the upstream **KiCadRoutingTools** repository (drandyhaas/KiCadRoutingTools, Haas 2026), the author recently addressed this exact pathology by adding **Rip-up Memory** (a lightweight detailed-routing history cost that penalizes blocking the cells where a previously ripped-up track ran), **Stub Proximity Avoidance** (penalizing A* traversal near unrouted pad stubs), and **Protected Nets** (py_router/protected_nets.py)[52].

Transferability to a 4-Layer PCB with Two Plane Layers

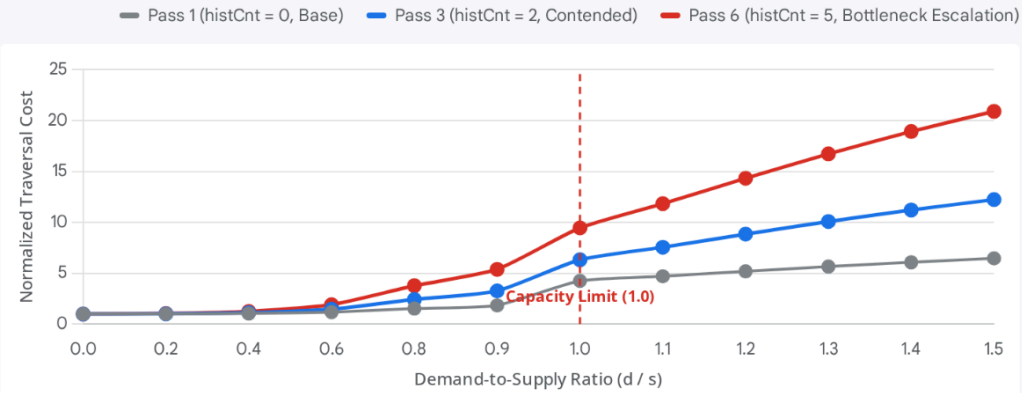
These mechanisms directly explain the campaign degradation observed in kicad-route. When kicad-route runs with rip-up disabled and promotes the 32 failed nets (including long inter-block buses) to the front of Round 2 on a fresh grid with zero congestion history, those long buses see an empty board and cut straight across dense pin fields along minimum-length octilinear paths. Because each $0.20\ \text{mm}$ track plus its $0.20\ \text{mm}$ clearance halo blocks a $0.60\ \text{mm}$ swath, a single early bus track seals off the local escape corridors of multiple short nets, causing open nets to jump from 32 to 59–64[2].

Persistent History Costs Prevent the Greedy Channel Capture of Net Reordering

1. Negotiated-Congestion Cost Escalation Across Routing Passes

$$cost_{tot} = 1 + w_1 \cdot cost_{cong} + w_2 \cdot (histCnt \cdot cost_{cong}) + w_4 \cdot cost_{ovfl}$$

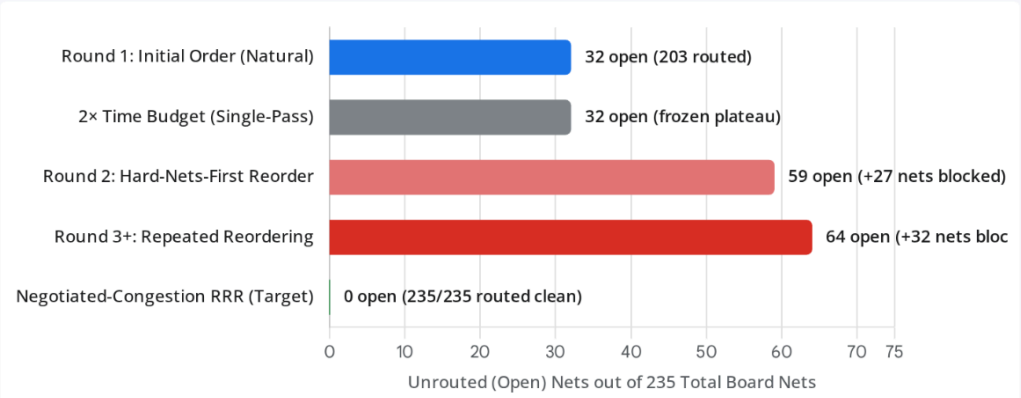
CUGR & PathFinder traversal cost as cell demand approaches and surpasses supply (capacity limit = 1.0). Uncontended cells cost ~1.0; bottleneck cells accumulate historical penalty scaling with `histCnt`.



2. Automotive PCB (235 Nets): Reordering Breakdown vs. History RRR

4-Layer Board (90×65 mm, 0.2 mm tracks, JLCPCB)

Promoting the 32 unrouted nets (dense eFuse resistors with 0.26 mm courtyard gaps) to the front of a fresh grid causes them to greedily capture bus corridors, stranding 27–32 previously routed nets (59–64 open).



Top: In negotiated-congestion routers (PathFinder, CUGR, TritonRoute), the logistic congestion cost rises as demand approaches supply, while the historical cost term (**cost_hist = histCnt × cost_cong**) scales up on every pass where a bottleneck cell experiences contention, eventually pricing out flexible buses so trapped intra-block nets can claim the corridor. Bottom: On the target 235-net automotive PCB, promoting the 32 failed nets to the front of Round 2 on a blank cost grid degrades open nets to 59–64 via greedy channel capture, whereas persisting spatial history costs across selective rip-up passes drives monotonic convergence.

Data sources: PathFinder Router (McMurchie & Ebeling, FPGA '95 / US Patent 7,134,112), CUGR Global Router (Liu et al., IEEE TCAD), TritonRoute (Kahng et al., IEEE TCAD), KiCad Automated Routing Tools (Haas)

In addition, a 4-layer PCB with In1=GND and In2=+3V3 offers two plane-specific optimizations that transfer directly to a 3D grid router. First, all GND and +3V3 pads should terminate immediately into In1 or In2 via local fanout vias and be skipped by the XY net router, preventing plane nets from consuming F.Cu or B.Cu channels[33]. Second, while In1 (GND) should remain a solid unbroken reference plane beneath high-speed and RF traces, In2 (+3V3) can legally carry short signal jumpers ($\leq$) to cross a dense bus when both F.Cu and B.Cu are saturated; in the A* graph, this is modeled by keeping In2 active with a high per-step base cost ($b_{\text{In2}} = 12 \times b_{\text{F.Cu}}$) and verifying via KiCad 10 DRC that the copper pour does not split into isolated islands (as checked in upstream KRT's measure_799_zone_feasibility.py)[33].

What This Would Mean for the In-House Tools

The single highest-payoff modification to kicad-route (~90 lines of Python/Rust) is replacing whole-board net-reordering campaigns with **Selective Rip-Up and Reroute over a Persistent 3D History Cost Grid** (history_cost[layer, x, y], initialized to 0.0)[18]. In Pass 1, route all nets in ascending HPWL (shortest and most pin-constrained nets first) with hard obstacle checking. For each net e_{open} that fails to route, immediately invoke a *negotiated A search** where grid cells occupied by existing routed tracks (and their $0.20\ \text{mm}$ clearance halos) are traversable at a finite penalty cell_cost = base_step_cost + history_cost[l, x, y] + ripup_override_cost (with ripup_override_cost set to $\approx 40 \times \text{base_step_cost}$, analogous to TritonRoute's object_cost)[18]. When negotiated A* returns a path for e_{open} , identify the specific $1\text{–}3$ existing nets whose tracks were crossed, **rip up only those $1\text{–}3$ conflicting nets** (leaving the other ~200 routed nets intact), increment history_cost[l, x, y] += h_step ($\approx 15 \times \text{base_step_cost}$, combining PathFinder's $h_n^{(k)}$, TritonRoute's marker_cost, and KRT's rip-up memory) on every collided cell and its clearance halo, commit e_{open} , and immediately re-route the displaced $1\text{–}3$ nets on the updated history_cost grid[18]. As a second-priority addition (~25 lines of code), add a **Stub-Proximity Halo Cost** (+5 * base_step_cost) on all grid cells within $0.60\ \text{mm}$ of every unrouted signal pad (masked off only for the source and target pads of the active net) so early nets do not hug unrouted pads and seal off their escape corridors[52].

4. Escape and Fanout

Prior Art for Escape Routing from Connector Pin Fields and Fine-Pitch Packages

When routing dense connectors (such as the automotive board's connector situated $0.36\ \text{mm}$ from mounting holes), QFN/BGA packages, or tightly packed IC clusters, sequential A* fails because early nets exiting the pin array wrap tightly around neighboring pins and block inner pins from escaping[2]. Prior art decouples **escape routing** into a dedicated

pre-stage before global routing[2]. For unordered pin fields, **Yan and Wong** (ICCAD 2009 / TCAD, *"A Correct Network Flow Model for Escape Routing"* and *"Correctly Modeling the Diagonal Capacity in Escape Routing"*) formulate escape routing on a directed tile graph where source vertices represent pins, relay vertices represent inter-pin routing tiles, and edge capacities equal the exact track supply $k = \lfloor (G_{\text{copper}} - s_{\text{clear}})/P_{\text{track}} \rfloor$ between adjacent pins[55]. Crucially, Yan and Wong demonstrated that standard orthogonal flow graphs overestimate diagonal capacity in grid and staggered pin arrays, introducing split-tile diagonal capacity edges so that a single min-cost maximum-flow execution guarantees 100% conflict-free pin escape to the component perimeter whenever a legal assignment exists[55]. When escaping pins belong to a multi-bit bus or connect to a common target macro, unordered escape introduces severe net crossings in the intermediate board area[55]. **Ordered Escape Routing (OER)**—developed by Luo et al., Sattar and Naveed (IEEE 2015, *"Ordered Escape Routing Using Network Flow and Optimization Model"*), and recent single-capacity GA/flow models (*Applied Sciences* 2026)—constructs four imaginary pin nodes (N-pinnode, E-pinnode, S-pinnode, W-pinnode) and eight around-box edges around each pin and replaces boundary terminal nodes with unit-capacity (cap = 1) imaginary edges[58]. Solving a Min-Cost Multi-Commodity Flow (MMCF) problem on this graph ensures that all escaping signals reach the component boundary in the exact topological sequence required by their target pads without crossing[55]. For multi-component buses, **Simultaneous Escape Routing** (Ozdal & Wong, TCAD) and **Negotiated-Congestion Escape Routing (NCER)** apply PathFinder's negotiated congestion directly on the pin-array tile graph to co-optimize escape ordering at both ends of a bus, solving complex pin-field instances that stall commercial routers such as Cadence Allegro[55].

How Routers Decide Escape Direction Before Global Routing

Both commercial PCB routers and open-source tools (such as Freerouting's pre-autoroute fanout pass and upstream KRT's test_860_qfn_escape_method_panel.py, test_fanout_rotated_gui.py, and test_gnd_vias_gui.py) determine escape direction prior to main routing using three deterministic heuristics[33]. First, **plane-net fanout precedes signal escape**: all GND and +3V3 pads are fanned out first via short dog-bone stubs and vias dropped away from the primary signal channel (or inward toward a thermal/ground pad), immediately freeing surface tracks between adjacent signal pins[33]. Second, for each signal pin p_i with target net centroid $\mathbf{p}_{\text{target}}$, candidate octilinear escape steps $\mathbf{v}_{\text{cand}}$ to the component courtyard boundary are scored by combining **cosine-similarity target alignment** with a hard keep-out/blockage veto (*Applied Sciences* 2026, Section 3.2)[60]:

$$S_{\text{escape}}(\mathbf{v}_{\text{cand}}) = w_{\text{dir}} \frac{(\mathbf{p}_{\text{target}} - \mathbf{p}_i) \cdot \mathbf{v}_{\text{cand}}}{\|\mathbf{p}_{\text{target}} - \mathbf{p}_i\| \|\mathbf{v}_{\text{cand}}\|} - w_{\text{obs}} \cdot \text{Blockage}$$

On the connector located $0.36\ \mathrm{mm}$ from the mounting holes, $\mathrm{BlockagePenalty}(\mathbf{v}_{\mathrm{cand}})$ vetoes outward escape toward the mounting hole, forcing the obstructed row of pins to escape inward through the inter-pin channels or via a board-side fanout via *before* the unobstructed inner row is routed[36]. Third, escaping pins along each component edge are sorted monotonically by the polar angle $\theta_i = \mathrm{atan2}(y_{\mathrm{target},i} - y_i, x_{\mathrm{target},i} - x_i)$ of their destination pads and assigned to non-crossing perimeter exit points on the component courtyard boundary so that initial fanout stubs never cross one another[52].

What This Would Mean for the In-House Tools

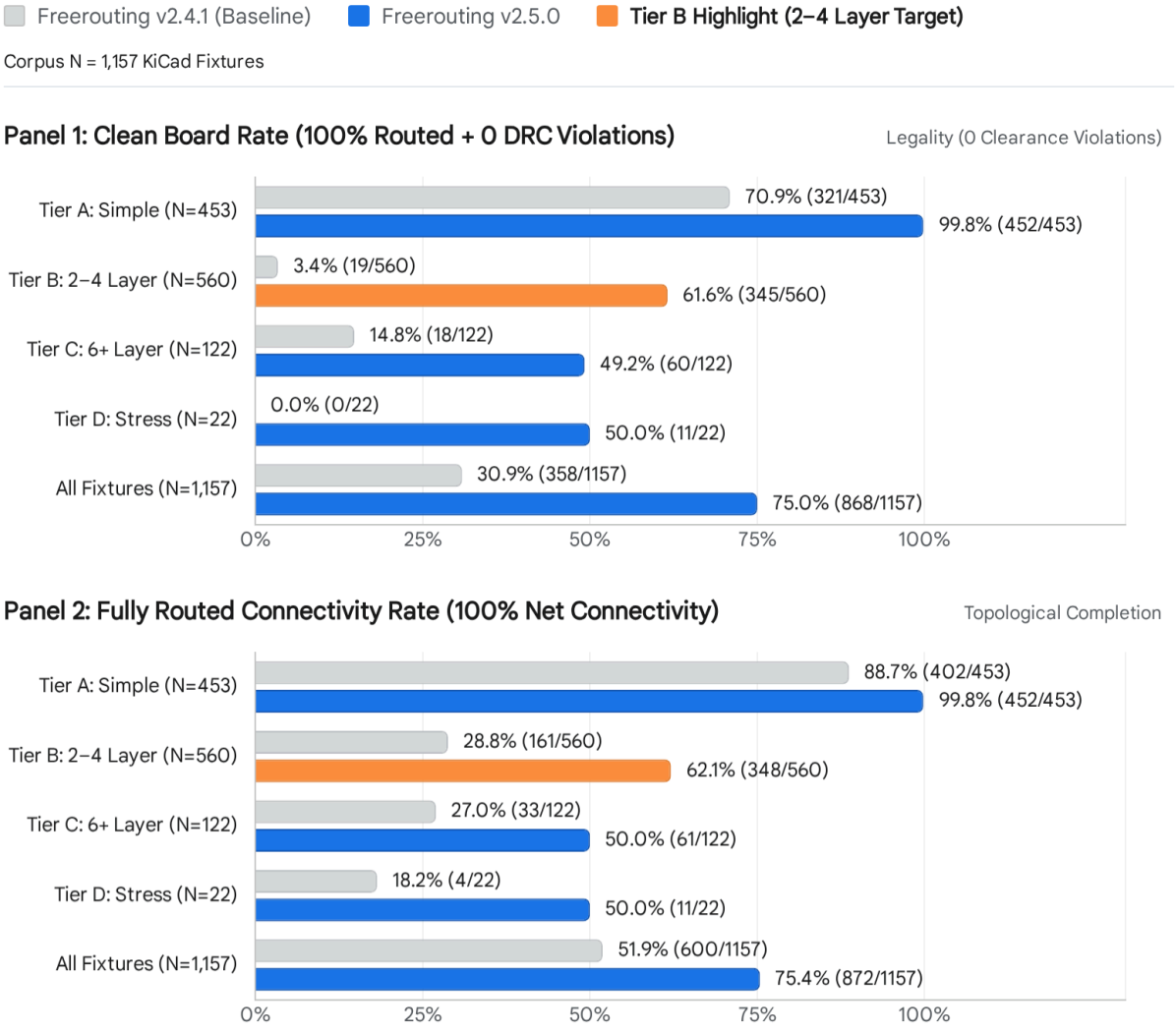
The highest-payoff, lowest-cost change (~60 lines of Python in kicad-route) is introducing a **Phase 0 Plane-Via Fanout and Monotonic Pin-Field Escape Pass** before running the main A* signal router[33]. First, kicad-route should iterate over all GND and +3V3 pads not already satisfied by a direct zone connection and drop a short ($\leq 0.8\ \mathrm{mm}$) dog-bone via to In1 or In2 in the octilinear direction that maximizes clearance to adjacent signal pads, marking those plane connections complete and locking them as protected copper[33]. Second, for any connector or IC with ≥ 8 pins, kicad-route should sort pins from the most mechanically obstructed row (such as the connector row $0.36\ \mathrm{mm}$ from the mounting holes) to the least obstructed row, order them monotonically by target angle θ_i, and route a short $1.0\text{–}1.5\ \mathrm{mm}$ escape stub to the footprint courtyard perimeter before global A* routing begins[36].

5. Benchmarks and Evaluation

Open PCB Placement and Routing Benchmarks

Four open-source PCB benchmark suites in native KiCad format are available to benchmark agent-ee and kicad-route and prevent regressions across algorithmic updates. The largest routing corpus is **PCBench** (originally curated by He et al., 2024, and expanded to **1,157 real-world open-source KiCad PCB designs** in github.com/freerouting/freerouting v2.5.0 / v3.0 alongside the 10-board DAC2020 KiCad benchmark suite in fixtures/)[24]. PCBench stratifies boards into four tiers: **Tier A** (453 simple 2-layer breakout boards), **Tier B** (560 standard 2–4 layer production boards—the exact structural peer group for the 4-layer automotive PCB), **Tier C** (122 dense 6+ layer boards), and **Tier D** (22 high-stress diagnostic boards)[36].

PCBench 1,157-Board Corpus Establishes Empirical Baselines for 2–4 Layer KiCad Routing



Empirical benchmark results across the 1,157-design PCBench open-source KiCad corpus comparing Freerouting v2.4.1 against v2.5.0 (which added plane-net skip guards, layer-init fixes, and deterministic post-route optimization). On **Tier B (560 standard 2–4 layer production boards)**, clean 0-DRC completion rose from 3.4% to 61.6%, providing a direct reference benchmark for the 4-layer automotive flow.

Data source: Freerouting Releases (PCBench Benchmark Corpus v2.4.1 vs. v2.5.0)

Complementing PCBench, **PCBWorld-Bench** (2026, arxiv.org/html/2607.05915v1) provides an open-source, KiCad-engine-grounded evaluation protocol across three datasets (D1 synthetic grid boards matching InstaDeep Jumanji's Connector environment at grid sizes $G \in \{10, 50, 100, 200, 500\}$, D2 synthetic 2-layer gridless boards, and D3 comprising 679 DRV-clean real-world KiCad 9/10 boards derived from PCBench with human reference

traces preserved)[24]. D3 is partitioned by pad count into **D3-A Small** (100 boards, 2–13 nets, 6–31 pads, 2–4 layers), **D3-B Medium** (287 boards, 5–42 nets, 31–100 pads, 2–4 layers), and **D3-C Large** (292 boards, 6–451 nets, 101–2,103 pads, 2–8 layers—matching the 235-net, 309-footprint scale of the in-house automotive board), and benchmarks rule-based routers (Freerouting, OrthoRoute, and KiCadRoutingTools / KRT) directly through the native KiCad DRC engine[24]. For placement benchmarking, **Cypress** (github.com/NVlabs/Cypress, ISPD 2025) releases an open-source PCB placement suite synthesized from real commercial designs with interfaces to open-source and commercial EDA routers[16]. Additionally, the upstream **KiCadRoutingTools stress suite** (drandyhaas/KiCadRoutingTools/tests/stress/) provides 33 manifest sets (manifest_set1.json through manifest_set30.json and manifest_set1monster.json through manifest_set3monster.json) alongside automated KiCad DRC grading scripts (grade_final.py, kicad_drc_compare.py, measure_routing.py)[35].

Standard Routability Metrics in Published Literature

Published PCB placement and routing papers report routability at two levels: per-board optimization metrics (used to guide iterative campaigns) and corpus-level completion rates (used to compare tool versions)[36]:

Metric Category	Metric Name & Mathematical Definition	Primary Source	Purpose in Autonomous Toolchain
Hard Sign-Off Feasibility	**Unconnected / Open Nets (** N_{open} **)** and **DRC Violation Count (** n_{drv} **)** via kicad-cli pcb drc [cite: 36, 54, 63]	PCBWorld-Bench (2026); PCBench [cite: 24, 36]	Primary binary gate: a board is **Clean** iff $N_{\text{open}} =$ and $n_{\text{drv}} =$ [36].
Non-Linear Completion Score	**Router Score (V2_CONTINUOUS** , $0–100$**)** [cite: 36]	Freerouting v2.5.0 (ItemRouteResult)[36]	Weights completing the **second half of connections higher than the first half**, preventing a campaign from trading 1 hard net for 2 easy nets[36].
Composite	$\Phi(s) =$	PCBWorld (2026,	Combines DRV

Optimization Potential	[cite: 63]	Eq. 1)[63]	penalty $f_{\text{drv}}(s)$, total track length $\ell(s)$, and via count $n_{\text{via}}(s)$ into a single scalar reward for campaign grading[63].
Lower-Bound Copper Efficiency	**Optimizer Score (V2_LOWER_BOUND,** 0–100**)** vs. $(L_{\min}, V_{\min}, B_{\min}$ [cite: 36]	Freerouting v2.5.0 (BatchOptimizer)[36]	Normalizes routed trace length $\ell(s)$ and via count n_{via} against placement-derived Manhattan minimum spanning tree lower bounds[36].
Placement Congestion Proxy	**Net Crossing Count (**NC**)** and **Top-10% RUDY (** $\frac{1}{\|C^*\|}\sum_{c\in C^*}\text{RUDY}$ **)** [cite: 16, 29]	Cypress (ISPD 2025); WireMask-EA (NeurIPS 2023)[16]	Fast pre-routing placement grade (<10 ms) that correlates directly with detailed routing completion on 2–4 layer boards[16].

What This Would Mean for the In-House Tools

The smallest concrete change to immediately improve campaign stability (~30 lines of Python in kicad-route's campaign orchestrator) is implementing Freerouting v2.5.0's **Best-Board Preservation guard** coupled with PCBWorld's lexicographical/potential scoring function $\Phi(s) = -(1000 \cdot N_{\text{open}} + 500 \cdot n_{\text{drv}} + \lambda_w(\ell(s)/L_{\min}) + \lambda_v n_{\text{via}})$ [36]. Before Pass 1, kicad-route should snapshot the best board state and update that snapshot only when a subsequent pass strictly improves $(N_{\text{open}}, n_{\text{drv}}, \Phi(s))$, automatically restoring the best-known board and terminating via a canary stagnation check if two consecutive passes fail to improve completion[36]. Second in priority (~1 day of engineering setup) is pulling a 20-board 4-layer slice from **PCBWorld-Bench D3-B / D3-C** (and NVlabs/Cypress for agent-ee) into continuous integration to track Clean Completion Rate (%), N_{open}, n_{drv}, and normalized wirelength $\ell(s)/L_{\min}$ across every commit[16].

Summary Roadmap: Ranked Architectural Changes by Payoff vs. Cost

Synthesizing the findings across all five questions, the stagnation at 203 of 235 nets on the 90 × 65 mm 4-layer automotive PCB can be resolved without altering the deterministic architecture of agent-ee or rewriting kicad-route from scratch. Implementing the following five minimal-diff changes in rank order directly eliminates the intra-block $0.26\ \mathrm{mm}$ courtyard traps and the inter-round bus starvation:

Rank	Target Tool	Concrete Architectural Change	Failure Mode Solved	Est. Effort	Expected Payoff
1	agent-ee (Intra-Block Packer)	**Pad-Edge Demand Halo + 2-Pin Net-Crossing Check**: Keep base courtyard gap at $0.16–0.25\ \mathrm{n}$ for fixed/RF/non-escaping edges, but enforce a $\geq$ **copper-to-copper gap** ($1\ \mathrm{track} + 2$) on pad-bearing edges of $>$ -pin ICs (eFuse) and	Eliminates the 0.26 mm courtyard traps around eFuse setting resistors while honoring the 0.36 mm mounting hole and 0.16 mm RF corridors.	~75 LOC Python	**Resolves ~20–25 of the 32 open nets** (all trapped intra-block nets).

		pass-through passives; evaluate $0^\circ/180^\circ$ passive flips to minimize intra-block net crossings[1].			
2	kicad-route (Grid Router Loop)	**Selective Rip-Up with Persistent 3D history_cost[l,x,y]**: Disable blind net-reordering on blank grids. Keep net order fixed; when a net fails, allow A* to cross existing tracks at ripup_override_cost + history_cost, rip up *only* the 1–3 crossed nets, increment history_cost on the collision cells+halos, and reroute	Eliminates campaign stagnation at 203/235 and prevents the 59–64 open-net regression caused by greedy bus channel capture.	~120 LOC Python/Rust	**Resolves remaining 7–12 inter-block / bus nets** and guarantees monotonic campaign progress.

		the displaced nets with Best-Board Preservation[18].			
3	kicad-route (Pre-Route Fanout)	**Phase 0 Plane-Via Fanout & Stub-Proximity Halo**: Drop dog-bone vias to In1 (GND) and In2 (+3V3) first and remove plane pins from XY routing; apply a $+5\times$ cost halo within $0.60\ \mathrm{mm}$ of all unrouted signal pads during A* search[33].	Prevents early tracks from boxing in connector and QFN pads before their own nets are routed.	~85 LOC Python/Rust	Eliminates pin-field boxing around the connector and dense ICs.
4	agent-ee (Macro Arranger)	**Asymmetric Pin-Flux Macro Halos**: Size each of the 4 sides of the 17–19 rigid functional	Reserves exact inter-block bus channels between functional blocks without	~50 LOC Python	Prevents long buses from starving between abutting functional blocks.

		blocks by $H(B_i, s) =$ (excluding GND/+3V3 nets) during macro packing[1].	wasting board area on zero-pin edges.		
5	kicad-route (Layer Graph)	**High-Cost Short Jumper Routing on In2 (+3V3)**: Allow A* to route short ($\leq$) signal bridges on In2 at $12\times$ base step cost, gated by KiCad 10 zone-island DRC verification[3][3].	Provides a 3rd escape dimension for unavoidable planar X-crossings when F.Cu and B.Cu are both saturated.	~40 LOC Python/Rust	Safety valve for the final 1–3 topological bus crossings.

Works cited

1. Channel spacing between the Macros | Forum for Electronics, https://www.edaboard.com/threads/channel-spacing-between-the-macros.230192/
2. A Pad-focused PCB Routing Algorithm Using Polygon-based, https://ieeexplore.ieee.org/iel7/6287639/6514899/10496097.pdf
3. FastGR : Global Routing on CPU-GPU with Heterogeneous Task, https://yibolin.com/publications/papers/ROUTE_DATE2022_Liu.pdf
4. Fast and Accurate Routing Demand Estimation for Efficient, https://past.date-conference.com/proceedings-archive/2007/DATE07/PDFFILES/08.7_1.PDF
5. AutoDMP: Automated DREAMPlace-based Macro Placement, https://d1qx31qr3h6wln.cloudfront.net/publications/AutoDMP.pdf
6. [PDF] Fast and Accurate Routing Demand Estimation for Efficient, https://www.semanticscholar.org/paper/Fast-and-Accurate-Routing-Demand-Est

imation-for-Spindler-Johannes/2ffc498231d2e7732828e80b05857f08619d8978
7. Global Placement with Deep Learning-Enabled Explicit Routability, https://www.cse.cuhk.edu.hk/~byu/papers/C112-DATE2021-DREAMPlace-Cong.pdf
8. Differentiable Cross-Stage Objectives for Chip Placement Optimization, https://arxiv.org/html/2608.13790v1
9. Copyright by Jiayuan He 2022 - The University of Texas at Austin, https://repositories.lib.utexas.edu/bitstreams/c4b6419d-840f-4bc1-b165-db2211cdd290/download
10. ycchen218/EDA-Congestion-Prediction - GitHub, https://github.com/ycchen218/EDA-Congestion-Prediction
11. Grafting Netlist Knowledge on Layout-Based Congestion Prediction, https://shipxu123.github.io/papers/J8-TCAD2025-Lay-Net.pdf
12. Efficient Quadratic Placement of VLSI Circuits, https://d-nb.info/990084132/34
13. Chip placement, routing, and timing analysis tools - Good GitHub, https://goodgithub.com/reports/chip-placement-routing-and-timing-analysis-tools.html
14. Optimizing Routability via Unified Placement and Routing Formulation, https://yibolin.com/publications/papers/PLACE_DAC2025_Chen.pdf
15. Towards a Reference Place and Route Flow for Academic Research, https://jics.org.br/ojs/index.php/JICS/article/download/648/426/2983
16. Cypress: VLSI-Inspired PCB Placement with GPU Acceleration, https://www.csl.cornell.edu/~zhiruz/pdfs/cypress-ispd2025.pdf
17. Apollo: Automated Routing-Informed Placement for Large-Scale, https://scopex-asu.github.io/files/publications/PD_ICCAD2025_Gu.pdf
18. TritonRoute-WXL: The Open Source Router with Integrated DRC, https://vlsicad.ucsd.edu/Publications/Journals/j136.pdf
19. GOALPlace: Begin with the End in Mind - arXiv, https://arxiv.org/html/2407.04579v1
20. A Multiple-objective ILP based Global Routing Approach for VLSI, https://www.uwspace.uwaterloo.ca/bitstreams/3fb3df78-9332-43c5-af74-5cf6ceeade33/download
21. yapnr/docs/references.md at main - GitHub, https://github.com/Studio-Fug/yapnr/blob/main/docs/references.md
22. Niansong Zhang, https://www.zzzdavid.tech/
23. Freerouting vs DeepPCB: A Better KiCad Autorouter, https://deeppcb.ai/freerouting-vs-deeppcb/
24. PCBWorld: A Benchmark Environment for Engine-Grounded PCB, https://arxiv.org/html/2607.05915v1
25. DeepPCB: AI PCB Routing for Engineering Teams, https://deeppcb.ai/
26. Automated PCB Design Using Artificial Intelligence (AI), https://pallavaggarwal.in/automated-pcb-design-using-ai/
27. Pure AI-Powered, Cloud-Native Printed Circuit Board Routing, https://news.ycombinator.com/item?id=21660365
28. Electronic design automation and PCB design systems - Good GitHub,

https://goodgithub.com/reports/electronic-design-automation-and-pcb-design-systems.html
29. Macro Placement by Wire-Mask-Guided Black-Box Optimization, https://proceedings.neurips.cc/paper_files/paper/2023/file/15d6717f8bb33b3a74df26ce1eee0b9a-Paper-Conference.pdf
30. Autodmp | PDF | Mathematical Optimization | Cluster Analysis - Scribd, https://www.scribd.com/document/740911142/autodmp
31. kicad-tools/docs/research/kicad-routing-tools-comparison.md at, https://github.com/rjwalters/kicad-tools/blob/main/docs/research/kicad-routing-tools-comparison.md
32. py_router/group_routing.py · KiCadRoutingTools · bus622-take5, https://thingatlas.com/-unclaimed/kicadroutingtools-1115830441/branches/bus622-take5/files/py_router/group_routing.py
33. KiCadRoutingTools · bus622-take5 - tests - ThingAtlas, https://thingatlas.com/-unclaimed/kicadroutingtools-1115830441/branches/bus622-take5/files/tests/gui_parity/test_movie_recorder.py
34. py_router/place_motion.py · KiCadRoutingTools · bus622-take5, https://thingatlas.com/-unclaimed/kicadroutingtools-1115830441/branches/bus622-take5/files/py_router/place_motion.py
35. tests/stress · KiCadRoutingTools · bus622-take5 | ThingAtlas, https://thingatlas.com/-unclaimed/kicadroutingtools-1115830441/branches/bus622-take5/files/tests/stress
36. Releases · freerouting/freerouting - GitHub, https://github.com/freerouting/freerouting/releases
37. PD Basic Notes | PDF | Design | Logic Synthesis - Scribd, https://www.scribd.com/document/1054227336/PD-Basic-Notes
38. Cadence Innovus User Guide v23.14 - IC Physical Design & EDA, https://studylib.net/doc/28007780/innovusug
39. PD Engineer Complete Interview Guide | PDF | Logic Gate - Scribd, https://www.scribd.com/document/1040518228/PD-Engineer-Complete-Interview-Guide
40. US7134112B1 - Incremental routing in integrated circuit design, https://patents.google.com/patent/US7134112B1/en
41. PathFinder: A Negotiation-Based Performance-Driven Router for, https://www.computer.org/csdl/proceedings-article/fpga/1995/25500111/12OmNxxvAHx
42. Why PCB Autorouters Failed: The Wrong Target - Quilter, https://www.quilter.ai/blog/pcb-autorouter-was-the-right-idea
43. Efficient Detailed Routing for FPGA Back-End Flow Using ... - MDPI, https://www.mdpi.com/2079-9292/11/14/2240
44. A SimPLR method for routability-driven placement, https://www.computer.org/csdl/proceedings-article/iccad/2011/06105307/12OmNyKa60q
45. Copyright by Hao Chen 2023 1 - The University of Texas at Austin, https://repositories.lib.utexas.edu/bitstream/handle/2152/121060/CHEN-DISSERTA

TION-2023.pdf?sequence=1&isAllowed=y
46. TritonRoute: The Open Source Detailed Router, https://vlsicad.ucsd.edu/Publications/Journals/j133.pdf
47. TritonRoute: The Open-Source Detailed Router - ResearchGate, https://www.researchgate.net/publication/342292568_TritonRoute_The_Open-Source_Detailed_Router
48. Asynchronous Reinforcement Learning Framework and Knowledge, https://z0ngqing.github.io/paper/tcad.pdf
49. Roadmap to version 2.2 · Issue #612 · freerouting/freerouting - GitHub, https://github.com/freerouting/freerouting/issues/612
50. Freerouting 2.2.3 seems to not recognize any layers in DSN file #676, https://github.com/freerouting/freerouting/issues/676
51. Performance Regression: High unrouted item count on ... - Issuehunt, https://oss.issuehunt.io/r/freerouting/freerouting/issues/608
52. KiCad Automated Routing Tools - Hackaday.io, https://hackaday.io/project/204891-kicad-automated-routing-tools
53. py_router/protected_nets.py · KiCadRoutingTools · bus622-take5, https://thingatlas.com/-unclaimed/kicadroutingtools-1115830441/branches/bus622-take5/files/py_router/protected_nets.py
54. KiCad Copilot MCP Server：配置与使用 - FunCoding, https://funcoding.ai/mcp/biosshot/kicad-copilot/
55. Correctly Modeling the Diagonal Capacity in Escape Routing, https://www.semanticscholar.org/paper/Correctly-Modeling-the-Diagonal-Capacity-in-Escape-Yan-Wong/7efa04804122865a14c90674374d535e2313bf1e
56. A correct network flow model for escape routing | Semantic Scholar, https://www.semanticscholar.org/paper/A-correct-network-flow-model-for-escape-routing-Yan-Wong/32f72c8568add2df77cf4d3a28895d5ab098a8be
57. Monotonic parallel and orthogonal routing for single-layer ball grid, https://www.semanticscholar.org/paper/Monotonic-parallel-and-orthogonal-routing-for-ball-Tomioka-Takahashi/2909a7637e8ceb5f592d7a7eb4db5f36f0451edd
58. Examples of the ordered escape routing on (a) grid pin array and (b), https://www.researchgate.net/figure/Examples-of-the-ordered-escape-routing-on-a-grid-pin-array-and-b-staggered-pin-array_fig1_325055836
59. Ordered escape routing using network flow and optimization model, https://ieeexplore.ieee.org/document/7081209/
60. A Genetic Algorithm-Based Optimization Method for Ordered Escape, https://www.mdpi.com/2076-3417/16/4/2010
61. Towards automated PCB routing: Leveraging machine learning and, https://dr.lib.iastate.edu/server/api/core/bitstreams/baa06fe6-541d-4f4a-888d-94f3083cd518/content
62. ISPD 2025 Program, https://ispd.cc/ispd2027/slides/ispd2025.html
63. PCBWorld: A Benchmark Environment for Engine-Grounded PCB, https://arxiv.org/html/2607.05915v4